Parallel computer architectures.

- Parallel systems are more complex than single-processor computers.
- Multiple CPUs must coordinate and synchronize their tasks.
- Parallel computers use different architectures.
- CPUs are the main part of parallel processing.

Flynn's Taxonomy

Two basic concepts:

Flynn ' s Taxonomy is needed to classify computer architectures based on how many instruction streams and data streams they can process at the same time

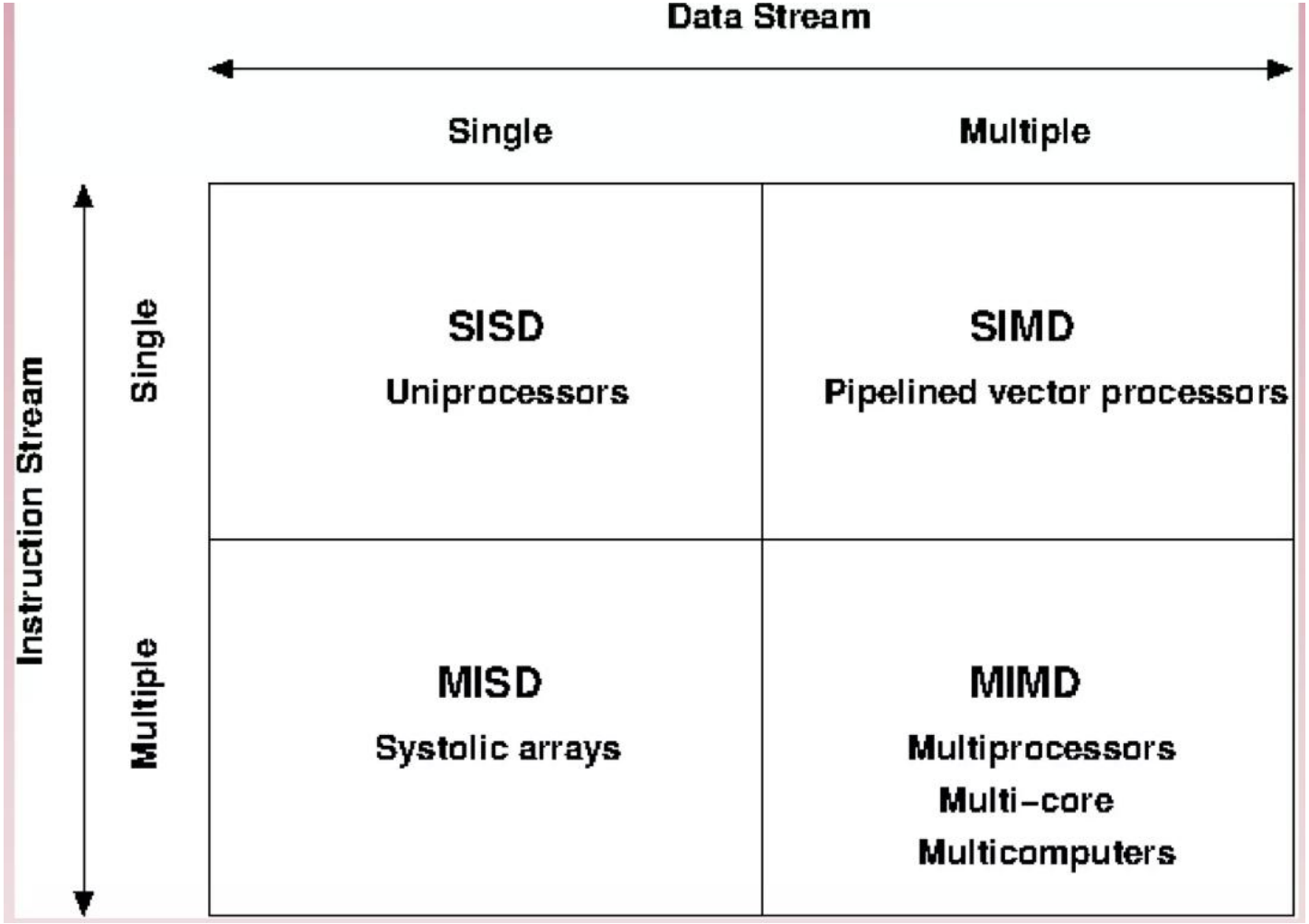
Instruction Streams: Sequences of instructions executed by a processor.

Data Streams: Sequences of data on which the instructions operate.

4 Categories:

- Single instruction stream, single data stream (SISD)
- Single instruction stream, multiple data stream (SIMD)
- Multiple instruction stream, single data stream (MISD)
- Multiple instruction stream, multiple data stream (MIMD)

Flynn's Taxonomy



Flynn's Taxonomy

Insrtuction Set	Data Streams	Name	Examples
1	1	SISD	Von Neumann Machine
1	Multiple	SIMD	Vector Super Computer
Multiple	1	MISD	Arguably None
Multiple	Multiple	MIMD	Multiprocessor, Multicomputer

Single instruction stream, single data stream (SISD)

Characteristics:

Single Control Unit (CU): Executes one instruction at a time.

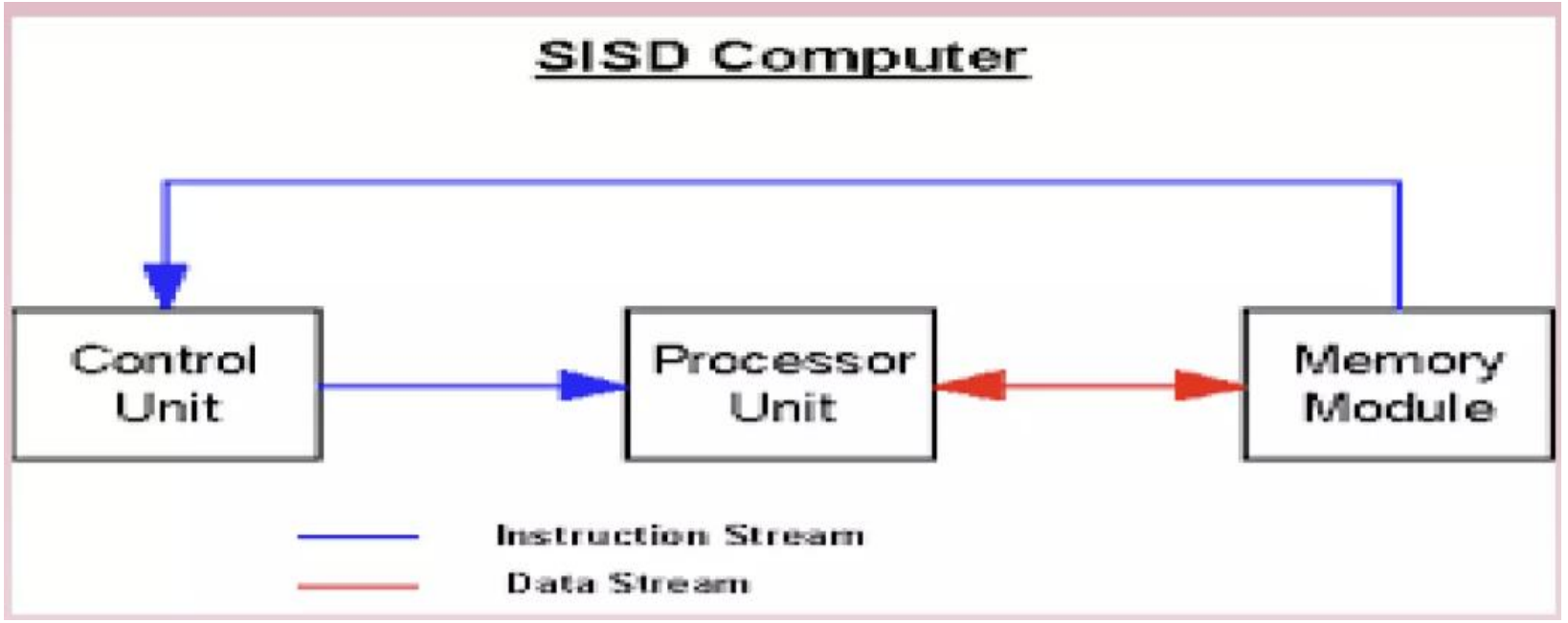
Single Data Stream: Operates on one piece of data at a time.

Sequential Execution: Processes tasks sequentially, with no parallelism.

Examples:

- Traditional uniprocessor systems such as early personal computers (e.g., Intel 8086).
- SISD computer using multiple functional units such as IBM 360/91, CDC Star-100, TI-ASC

Single instruction stream, single data stream (SISD)



Single instruction stream, single data stream (SISD)

Uses:

- We do not need parallel processing
- We require sequential execution of tasks

Limitations

Poor scalability: As computational demands grow, SISD systems struggle to keep pace due to their lack of parallelism.

Single instruction stream, Multiple data stream (SIMD)

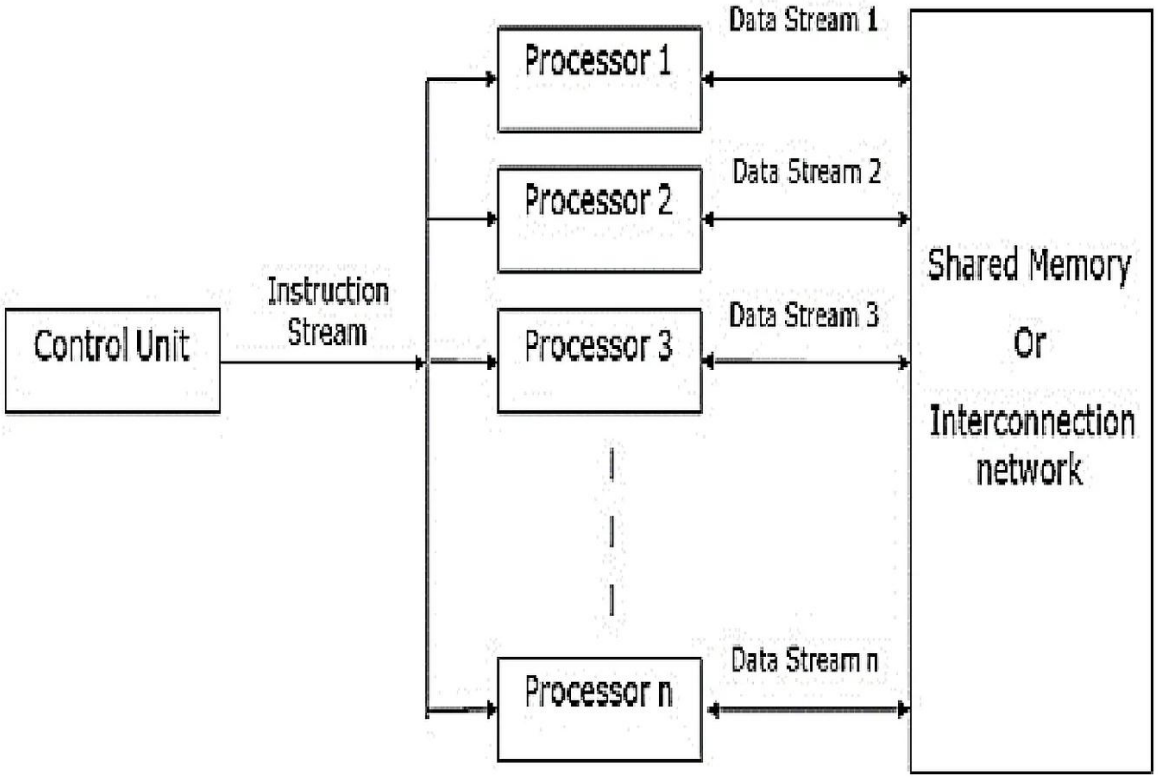
Characteristics:

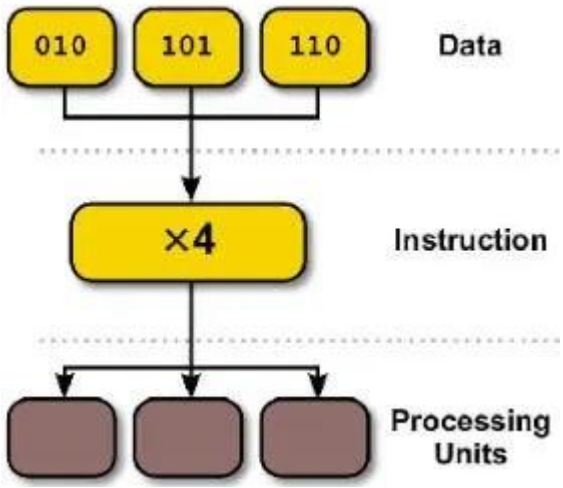
- **Single Control Unit:** Executes a single instruction across multiple data elements simultaneously.
- **Data-Level Parallelism (DLP):** Exploits parallelism in datasets, making it highly efficient for tasks with repetitive operations.
- **Hardware Implementation:** Typically includes vector processors or specialized instruction sets in modern CPUs and GPUs.

Examples:

- **Vector Processors:** Early examples include the **Cray-1** supercomputer.
- **Modern CPUs and GPUs:** Technologies like Intel's AVX (Advanced Vector Extensions) and NVIDIA's CUDA leverage SIMD principles.

Single instruction stream, Multiple data stream (SIMD)





Single instruction stream, Multiple data stream (SIMD)

Uses:

- **Image and video processing:** Applying the same operation to multiple frames.
- **Scientific simulations:** Calculations on large datasets, such as matrix multiplication.
- **Machine learning:** ANN operations benefit significantly from SIMD parallelism.

Limitations:

- **Requires uniformity:** Tasks must perform the same operation on all data elements, which limits flexibility.
- Tasks that have a high degree of data dependency may not benefit as much from SIMD parallelism.

Multiple instruction stream, Single data stream (MISD)

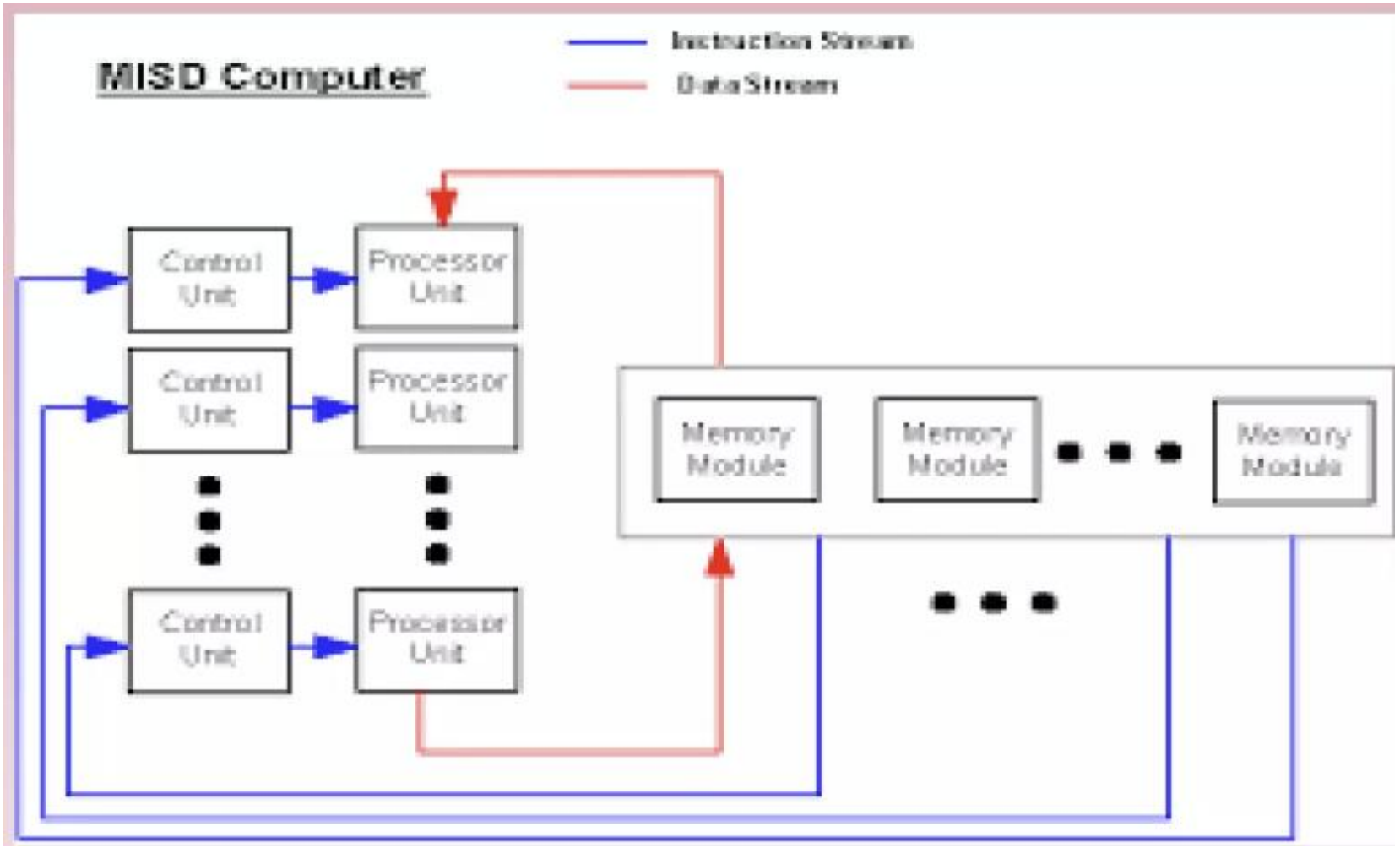
Characteristics:

- There are ***n*** processor elements.
- Each processor elements receives distinct instructions to execute on the same data stream and its derivatives.
- The output of one processor element becomes the input of the next processor element in the series.

Examples:

There is no practical example of MISD yet.

Multiple instruction stream, Single data stream (MISD)



Multiple instruction stream, Single data stream (MISD)

Limitations:

- Limited scalability and efficiency for general-purpose computing.
- Challenging to find real-world problems that fit this model.

Multiple instruction stream, Multiple data stream (MIMD)

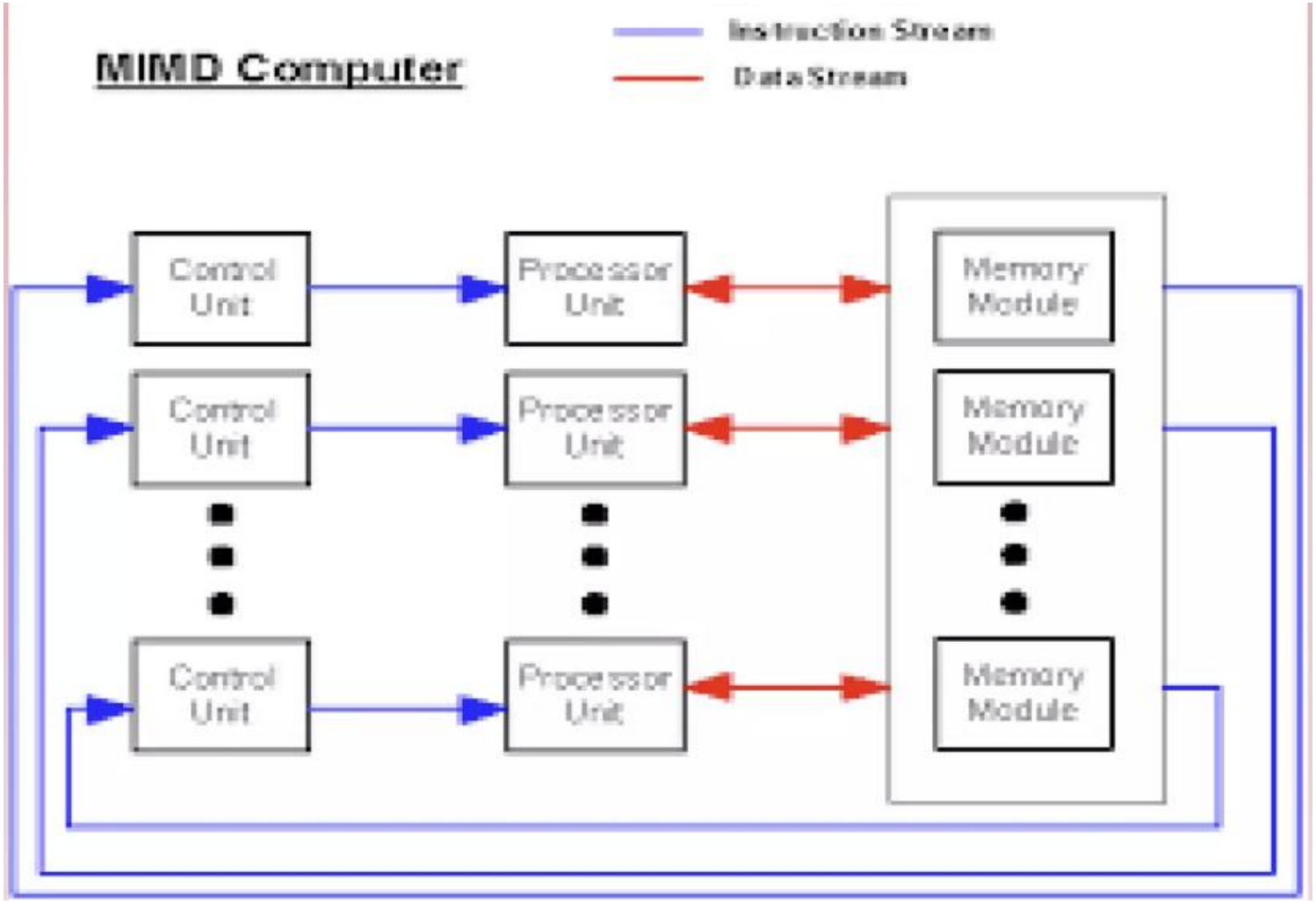
Characteristics

- **Multiple Control Units:** Execute different instructions independently.
- **Multiple Data Streams:** Each processor operates on its own dataset.
- **Task-Level Parallelism:** Suitable for complex and heterogeneous workloads.

Examples

- **Multicore Processors:** Modern CPUs with multiple cores (e.g., AMD Ryzen, Intel Core).
- **Distributed Systems:** Clusters and cloud computing infrastructures.
- **GPUs:** GPUs employ MIMD principles for specific tasks.
- **Supercomputers:** High-performance supercomputers, such as the IBM Summit.

Multiple instruction stream, Multiple data stream (MIMD)



Multiple instruction stream, Multiple data stream (MIMD)

Uses:

- Multithreaded and multi tasking applications
- High performance computing such as big data analytics, cryptography, etc.
- Training complex AI models.

Limitations

- **Higher complexity:** Requires sophisticated synchronization mechanisms.
- **Power consumption:** Running multiple processors simultaneously demands significant energy.